

Praktikum 8: Virtual Memory

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

INFORMASI UMUM

Deskripsi Praktikum

Praktikum ini membahas konsep dan implementasi Virtual Memory pada sistem operasi Linux. Mahasiswa akan mempelajari berbagai aspek virtual memory mulai dari konsep dasar hingga implementasi algoritma page replacement, thrashing detection, dan kernel memory allocation melalui pemrograman C dan eksperimen langsung pada sistem Ubuntu.

Software yang Dibutuhkan

```
# Cek dan install tools yang diperlukan
sudo apt-get update
sudo apt-get install build-essential
sudo apt-get install gcc
sudo apt-get install gdb
sudo apt-get install valgrind
sudo apt-get install zlib1g-dev
sudo apt-get install sysstat
```

TUJUAN PEMBELAJARAN

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. **Memahami konsep virtual memory** dan perbedaannya dengan physical memory
 2. **Mengimplementasikan dan menganalisis** algoritma page replacement (FIFO, LRU, Clock)
 3. **Mensimulasikan demand paging** dan memahami mekanisme page fault
 4. **Mendemonstrasikan Copy-on-Write** pada proses fork
 5. **Mendeteksi dan mencegah thrashing** menggunakan working set model
 6. **Menganalisis memory compression** sebagai alternatif swapping
 7. **Memahami kernel memory allocation** dengan Buddy System dan Slab Allocator
 8. **Memonitor dan menganalisis** virtual memory pada sistem Linux real
-

PERSIAPAN PRAKTIKUM

Langkah 1: Setup Environment

```
# Buat direktori kerja
mkdir -p ~/praktikum-vm
cd ~/praktikum-vm

# Buat subdirektori untuk setiap bagian
mkdir {background,demand-paging,cow,page-
replacement,frames,thrashing,compression,kernel-
alloc,considerations,examples}

# Cek system info
echo "=== System Information ==="
uname -a
free -h
getconf PAGE_SIZE
```

Langkah 2: Verifikasi Tools

```
# Cek GCC
gcc --version

# Cek apakah bisa compile C program
cat > test.c << 'EOF'
#include <stdio.h>
int main() {
    printf("Setup OK!\n");
    return 0;
}
EOF

gcc -o test test.c
./test
rm test test.c
```

MATERI PRAKTIKUM

1. BACKGROUND

1.1 Penjelasan Konsep

Virtual Memory adalah teknik manajemen memori yang memungkinkan program menggunakan lebih banyak memori dari yang tersedia secara fisik (RAM). Sistem operasi "menipu" program seolah-olah memori yang tersedia sangat besar, padahal sebagian data disimpan di disk (swap space).

Konsep Penting:

- **Physical Memory (RAM):** Memori fisik yang terpasang di komputer
- **Virtual Address Space:** Ruang alamat memori yang dilihat oleh program
- **Page:** Unit terkecil memori virtual (biasanya 4KB di Linux)
- **Frame:** Unit terkecil physical memory yang berkorespondensi dengan page
- **Page Table:** Struktur data yang memetakan virtual address ke physical address
- **Swap Space:** Area di disk yang digunakan sebagai "overflow" untuk RAM

Keuntungan Virtual Memory:

1. Program dapat lebih besar dari physical memory
2. Memory protection antar proses
3. Sharing memory antar proses
4. Efficient process creation (Copy-on-Write)

1.2 Praktik: Memeriksa Konfigurasi Memory System

Langkah 1: Buka terminal dan periksa informasi memori

```
cd ~/praktikum-vm/background

# Melihat total RAM dan swap
free -h

# Output contoh:
#               total        used        free      shared  buff/cache
available
# Mem:           7.7G         2.1G         3.2G          200M         2.4G
5.1G
# Swap:           2.0G           0B         2.0G

# Melihat informasi detail tentang memori
cat /proc/meminfo | grep -E "MemTotal|MemFree|SwapTotal|SwapFree"

# Melihat ukuran page size
getconf PAGE_SIZE
# Output: 4096 (4KB)
```

Interpretasi Output:

- **MemTotal**: Total RAM yang tersedia
- **MemFree**: RAM yang benar-benar kosong
- **MemAvailable**: RAM yang bisa digunakan (termasuk cache yang bisa dibebaskan)
- **SwapTotal**: Total swap space
- **PAGE_SIZE**: Ukuran satu page (biasanya 4096 bytes = 4KB)

Langkah 2: Buat program untuk melihat virtual address space

```
nano vm_basic.c
```

Ketik kode berikut:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100; // Segment data
int uninit_var;       // Segment BSS

int main() {
    int stack_var = 10; // Stack
    int *heap_var = (int*)malloc(sizeof(int)); // Heap
    *heap_var = 20;

    printf("=== Virtual Address Space ===\n");
    printf("Alamat variabel global (data): %p\n", (void*)&global_var);
    printf("Alamat variabel uninitialized (BSS): %p\n",
(void*)&uninit_var);
    printf("Alamat variabel stack: %p\n", (void*)&stack_var);
    printf("Alamat variabel heap: %p\n", (void*)heap_var);
    printf("Alamat fungsi main (text): %p\n", (void*)main);

    printf("\nPage Size: %ld bytes\n", sysconf(_SC_PAGE_SIZE));
    printf("Process ID: %d\n", getpid());

    printf("\nTekan Enter untuk melihat memory maps...\n");
    getchar();

    // Membuka memory maps
    char command[100];
    sprintf(command, "cat /proc/%d/maps", getpid());
    system(command);

    free(heap_var);
    return 0;
}
```

Langkah 3: Compile dan jalankan

```
gcc -o vm_basic vm_basic.c
./vm_basic
```

Langkah 4: Analisis Output

Perhatikan:

- Alamat memori berbeda untuk setiap segment (text, data, BSS, heap, stack)
- Alamat dalam format hexadecimal
- Stack biasanya di alamat tinggi, heap di alamat rendah-menengah
- Memory maps menunjukkan semua region yang dimapping

Tugas Eksplorasi:

1. Jalankan program beberapa kali, apakah alamat berubah? Mengapa?
 2. Bandingkan dengan `/proc/self/maps` untuk melihat memory layout proses lain
 3. Hitung berapa banyak page yang digunakan oleh setiap segment
-

2. DEMAND PAGING

2.1 Penjelasan Konsep

Demand Paging adalah teknik lazy loading dimana page hanya dimuat ke memori ketika dibutuhkan (terjadi page fault), bukan dimuat semua di awal program berjalan.

Proses Demand Paging:

1. Program mencoba mengakses alamat virtual
2. MMU (Memory Management Unit) memeriksa page table
3. Jika page **tidak ada** di memori (valid bit = 0) → **Page Fault**
4. Page fault handler (OS) dipanggil
5. OS memuat page dari disk ke RAM
6. Page table diupdate (valid bit = 1, frame number diisi)
7. Instruksi yang menyebabkan page fault diulang
8. Program melanjutkan eksekusi

Keuntungan:

- Startup program lebih cepat
- Hanya page yang digunakan yang dimuat
- Lebih efisien untuk program besar

Kerugian:

- Overhead saat page fault
- Perlu swap space di disk

2.2 Praktik: Simulasi Demand Paging

Langkah 1: Pindah ke direktori dan buat file

```
cd ~/praktikum-vm/demand-paging
nano demand_paging.c
```

Langkah 2: Ketik kode simulasi (lihat kode lengkap di bawah)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define PAGE_SIZE 4096
#define NUM_PAGES 10
#define PHYSICAL_FRAMES 5

typedef struct {
    int page_number;
    int frame_number;
    int valid; // 0 = not in memory, 1 = in memory
    int referenced;
    int modified;
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
} Frame;

PageTableEntry page_table[NUM_PAGES];
Frame physical_memory[PHYSICAL_FRAMES];
int page_faults = 0;

void init_system() {
    for (int i = 0; i < NUM_PAGES; i++) {
        page_table[i].page_number = i;
        page_table[i].frame_number = -1;
        page_table[i].valid = 0;
        page_table[i].referenced = 0;
        page_table[i].modified = 0;
    }

    for (int i = 0; i < PHYSICAL_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].page_number = -1;
    }
}
```

```

int find_free_frame() {
    for (int i = 0; i < PHYSICAL_FRAMES; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
    return -1;
}

void load_page(int page_num) {
    printf("\n[PAGE FAULT] Page %d tidak ada di memori!\n", page_num);
    page_faults++;

    printf(" → Memuat page %d dari disk...\n", page_num);

    int frame = find_free_frame();

    if (frame == -1) {
        printf(" → Memori penuh! Perlu page replacement\n");
        return;
    }

    // Simulasi loading delay
    for (long i = 0; i < 100000000; i++);

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;

    page_table[page_num].valid = 1;
    page_table[page_num].frame_number = frame;

    printf(" → Page %d dimuat ke Frame %d\n", page_num, frame);
}

void access_page(int page_num, char access_type) {
    printf("\n>>> Mengakses Page %d (tipe: %c)\n", page_num, access_type);

    if (!page_table[page_num].valid) {
        load_page(page_num);
    } else {
        printf(" → Page %d sudah ada di Frame %d (HIT!)\n",
            page_num, page_table[page_num].frame_number);
    }

    page_table[page_num].referenced = 1;
    if (access_type == 'W') {
        page_table[page_num].modified = 1;
        printf(" → Page %d di-modified (dirty bit = 1)\n", page_num);
    }
}

void print_status() {
    printf("\n===== STATUS SISTEM =====\n");
    printf("Page Table:\n");
}

```

```

printf("Page | Valid | Frame | Ref | Mod\n");
printf("-----|-----|-----|-----|-----\n");
for (int i = 0; i < NUM_PAGES; i++) {
    printf("  %2d |   %d   |   %2d |   %d   |   %d\n",
        page_table[i].page_number,
        page_table[i].valid,
        page_table[i].frame_number,
        page_table[i].referenced,
        page_table[i].modified);
}

printf("\nPhysical Memory (Frames):\n");
printf("Frame | Occupied | Page\n");
printf("-----|-----|-----\n");
for (int i = 0; i < PHYSICAL_FRAMES; i++) {
    printf("  %2d |   %d   |   %2d\n",
        i,
        physical_memory[i].occupied,
        physical_memory[i].page_number);
}

printf("\nTotal Page Faults: %d\n", page_faults);
printf("=====\n");
}

int main() {
    printf("=== SIMULASI DEMAND PAGING ===\n");
    printf("Total Pages: %d\n", NUM_PAGES);
    printf("Physical Frames: %d\n", PHYSICAL_FRAMES);
    printf("Page Size: %d bytes\n\n", PAGE_SIZE);

    init_system();

    printf("Tekan Enter untuk mulai simulasi...\n");
    getchar();

    // Simulasi akses memori
    access_page(0, 'R');
    print_status();
    printf("\nTekan Enter untuk lanjut...\n");
    getchar();

    access_page(1, 'R');
    print_status();
    printf("\nTekan Enter untuk lanjut...\n");
    getchar();

    access_page(2, 'W');
    print_status();
    printf("\nTekan Enter untuk lanjut...\n");
    getchar();

    access_page(0, 'R'); // Hit!
    print_status();

```



```
printf("\nTekan Enter untuk lanjut...\n");
getchar();

access_page(3, 'R');
access_page(4, 'W');
print_status();
printf("\nTekan Enter untuk lanjut...\n");
getchar();

access_page(5, 'R');
print_status();

printf("\n=== KESIMPULAN ===\n");
printf("- Page fault terjadi saat page belum ada di memori\n");
printf("- Page hit terjadi saat page sudah ada di memori\n");
printf("- Modified bit menandai page yang perlu ditulis kembali\n");
printf("- Demand paging menunda loading hingga benar-benar
dibutuhkan\n");

return 0;
}
```

Langkah 3: Compile dan jalankan

```
gcc -o demand_paging demand_paging.c
./demand_paging
```

Langkah 4: Analisis

Perhatikan:

- Kapan terjadi page fault vs page hit
- Bagaimana page table dan physical memory berubah
- Berapa total page faults yang terjadi
- Apa yang terjadi ketika memori penuh?

Pertanyaan Diskusi:

1. Mengapa page 0 ketika diakses kedua kali tidak menyebabkan page fault?
 2. Apa dampak modified bit pada performance?
 3. Bagaimana jika physical frames lebih sedikit dari yang dibutuhkan?
-

3. COPY-ON-WRITE (COW)

3.1 Penjelasan Konsep

Copy-on-Write (COW) adalah optimasi dimana ketika proses di-fork, child process tidak langsung mendapat salinan memori parent. Mereka berbagi page yang sama (read-only) sampai salah satu proses **menulis** ke page tersebut.

Mekanisme:

1. Process A melakukan fork() → Process B dibuat
2. Awalnya A dan B berbagi page yang sama
3. Page ditandai sebagai read-only untuk kedua proses
4. Ketika A atau B mencoba **menulis**, terjadi page fault
5. OS membuat copy dari page tersebut
6. Page yang baru diberikan ke proses yang menulis
7. Kedua proses sekarang punya page terpisah

Keuntungan:

- Fork sangat cepat (tidak perlu copy memory)
- Hemat memori (hanya copy page yang dimodifikasi)
- Efisien untuk exec setelah fork

Contoh Penggunaan:

- Shell menjalankan command (fork + exec)
- Banyak process yang read-only shared libraries

3.2 Praktik: Demonstrasi Copy-on-Write

Langkah 1: Buat program COW

```
cd ~/praktikum-vm/cow
nano copy_on_write.c
```

Langkah 2: Ketik kode berikut

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define ARRAY_SIZE 1000000

void print_memory_info(const char* label, int* array) {
    printf("\n[%s]\n", label);
    printf("PID: %d\n", getpid());
```

```

printf("Alamat array: %p\n", (void*)array);
printf("Nilai array[0]: %d\n", array[0]);

char command[200];
sprintf(command, "cat /proc/%d/status | grep -E
'VmSize|VmRSS|VmData'",
        getpid());
system(command);
}

int main() {
    printf("=== DEMONSTRASI COPY-ON-WRITE ===\n\n");

    // Alokasi memori besar
    int* large_array = (int*)malloc(ARRAY_SIZE * sizeof(int));
    if (large_array == NULL) {
        perror("malloc");
        return 1;
    }

    // Inisialisasi array
    printf("Parent: Menginisialisasi array dengan %d elemen...\n",
ARRAY_SIZE);
    for (int i = 0; i < ARRAY_SIZE; i++) {
        large_array[i] = i;
    }

    print_memory_info("PARENT - SEBELUM FORK", large_array);

    printf("\n\nTekan Enter untuk fork...\n");
    getchar();

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(large_array);
        return 1;
    }

    if (pid == 0) {
        // === CHILD PROCESS ===
        printf("\n\n===== CHILD PROCESS =====\n");
        sleep(1); // Beri waktu parent print dulu

        print_memory_info("CHILD - SETELAH FORK (Sebelum Write)",
large_array);

        printf("\n\nChild: Array masih berbagi dengan parent (COW)\n");
        printf("Child: Tekan Enter untuk MENULIS ke array (trigger
COW)...\n");
        getchar();

        // MENULIS - ini akan trigger copy-on-write

```

```

    printf("\nChild: Menulis ke array...\n");
    for (int i = 0; i < ARRAY_SIZE; i++) {
        large_array[i] = i * 2; // Modifikasi
    }
    printf("Child: Selesai menulis.\n");

    print_memory_info("CHILD - SETELAH WRITE", large_array);

    printf("\nChild: Array sekarang terpisah dari parent!\n");
    printf("Child: Nilai array[0] = %d (sudah dimodifikasi)\n",
large_array[0]);

    printf("\nChild: Tekan Enter untuk exit...\n");
    getchar();

    free(large_array);
    exit(0);

} else {
    // === PARENT PROCESS ===
    printf("\n\n===== PARENT PROCESS =====\n");

    print_memory_info("PARENT - SETELAH FORK", large_array);

    printf("\n\nParent: Menunggu child melakukan operasi...\n");
    printf("Parent: Nilai array[0] = %d (tidak berubah)\n",
large_array[0]);

    // Tunggu child selesai
    wait(NULL);

    printf("\n\nParent: Child telah selesai\n");
    print_memory_info("PARENT - SETELAH CHILD SELESAI", large_array);
    printf("Parent: Nilai array[0] = %d (tetap tidak berubah!)\n",
        large_array[0]);

    printf("\n=== KESIMPULAN ===\n");
    printf("1. Setelah fork, memory tidak langsung di-copy\n");
    printf("2. Parent dan child berbagi page yang sama (COW)\n");
    printf("3. Ketika child menulis, page baru dibuat (copy
terjadi)\n");
    printf("4. Setelah write, parent dan child punya memory
terpisah\n");
    printf("5. Ini menghemat waktu dan memori!\n");

    free(large_array);
}

return 0;
}

```

Langkah 3: Compile dan jalankan

```
gcc -o copy_on_write copy_on_write.c
./copy_on_write
```

Langkah 4: Observasi

Perhatikan:

- Memory usage sebelum dan setelah fork
- Memory usage sebelum dan setelah child menulis
- Alamat array sama tetapi isi bisa berbeda

Eksperimen Tambahan:

```
# Monitor memory real-time di terminal terpisah
watch -n 1 "ps aux | grep copy_on_write | grep -v grep"
```

Pertanyaan Diskusi:

1. Berapa memory yang dihemat dengan COW dibanding copy langsung?
2. Apa yang terjadi jika child tidak pernah menulis ke memory?
3. Bagaimana COW membantu performance shell command execution?

4. PAGE REPLACEMENT

4.1 Penjelasan Konsep

Ketika terjadi page fault dan **memori penuh**, OS harus memilih page mana yang akan **di-replace** (diganti). Pemilihan page yang buruk dapat menyebabkan banyak page fault.

Algoritma Page Replacement:

1. FIFO (First-In-First-Out)

- Page yang paling lama di memori diganti
- Sederhana tapi tidak optimal
- Dapat mengalami Belady's Anomaly

2. LRU (Least Recently Used)

- Page yang paling lama tidak digunakan diganti
- Mendekati optimal
- Sulit diimplementasikan secara efisien

3. Optimal (Teoritis)

- Ganti page yang tidak akan digunakan dalam waktu terlalu lama
- Tidak bisa diimplementasikan (butuh prediksi masa depan)
- Digunakan sebagai benchmark

4. Clock / Second Chance

- Variasi FIFO dengan reference bit
- Efisien dan cukup baik
- Digunakan banyak OS modern

Belady's Anomaly: Fenomena dimana menambah jumlah frame malah menyebabkan **lebih banyak** page faults (hanya terjadi pada FIFO).

4.2 Praktik: Implementasi Page Replacement Algorithms

Langkah 1: Buat program simulasi

```
cd ~/praktikum-vm/page-replacement
nano page_replacement.c
```

Langkah 2: Ketik kode lengkap (kode cukup panjang, pastikan lengkap)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

#define MAX_FRAMES 10
#define MAX_PAGES 20

typedef struct {
    int page_number;
    int referenced;
    int modified;
    int load_time;
    int last_access_time;
} Frame;

// ===== FIFO ALGORITHM =====
int fifo_replacement(Frame frames[], int num_frames, int current_time) {
    int oldest_index = 0;
    int oldest_time = frames[0].load_time;

    for (int i = 1; i < num_frames; i++) {
        if (frames[i].load_time < oldest_time) {
            oldest_time = frames[i].load_time;
            oldest_index = i;
        }
    }

    printf(" [FIFO] Mengganti page %d (loaded at time %d)\n",
           frames[oldest_index].page_number, oldest_time);
    return oldest_index;
}
```

```
// ===== LRU ALGORITHM =====
int lru_replacement(Frame frames[], int num_frames, int current_time) {
    int lru_index = 0;
    int lru_time = frames[0].last_access_time;

    for (int i = 1; i < num_frames; i++) {
        if (frames[i].last_access_time < lru_time) {
            lru_time = frames[i].last_access_time;
            lru_index = i;
        }
    }

    printf(" [LRU] Mengganti page %d (last access at time %d)\n",
           frames[lru_index].page_number, lru_time);
    return lru_index;
}

// ===== CLOCK ALGORITHM =====
int clock_replacement(Frame frames[], int num_frames, int *clock_hand) {
    while (1) {
        if (frames[*clock_hand].referenced == 0) {
            int victim = *clock_hand;
            *clock_hand = (*clock_hand + 1) % num_frames;
            printf(" [CLOCK] Mengganti page %d di frame %d\n",
                   frames[victim].page_number, victim);
            return victim;
        }

        // Give second chance
        printf(" [CLOCK] Page %d di frame %d mendapat second chance\n",
               frames[*clock_hand].page_number, *clock_hand);
        frames[*clock_hand].referenced = 0;
        *clock_hand = (*clock_hand + 1) % num_frames;
    }
}

void print_frames(Frame frames[], int num_frames) {
    printf(" Frames: [");
    for (int i = 0; i < num_frames; i++) {
        if (frames[i].page_number == -1) {
            printf(" - ");
        } else {
            printf(" %d%s", frames[i].page_number,
                   frames[i].referenced ? "R" : "");
        }
        if (i < num_frames - 1) printf(" |");
    }
    printf(" ]\n");
}

void simulate_algorithm(const char* name, int page_sequence[], int
seq_length,
                       int num_frames, int algo_type) {
```

```

printf("\n\n===== ALGORITMA %s =====\n", name);
printf("Number of Frames: %d\n", num_frames);
printf("Page Reference String: ");
for (int i = 0; i < seq_length; i++) {
    printf("%d ", page_sequence[i]);
}
printf("\n\n");

Frame frames[MAX_FRAMES];
for (int i = 0; i < num_frames; i++) {
    frames[i].page_number = -1;
    frames[i].referenced = 0;
    frames[i].modified = 0;
    frames[i].load_time = 0;
    frames[i].last_access_time = 0;
}

int page_faults = 0;
int clock_hand = 0;

for (int time = 0; time < seq_length; time++) {
    int page = page_sequence[time];
    printf("\n--- Time %d: Mengakses Page %d ---\n", time + 1, page);

    // Check if page is in frames
    int found_index = -1;
    for (int i = 0; i < num_frames; i++) {
        if (frames[i].page_number == page) {
            found_index = i;
            break;
        }
    }

    if (found_index != -1) {
        // Page hit
        printf("  ✓ Page Hit! Page %d ada di frame %d\n", page,
found_index);
        frames[found_index].referenced = 1;
        frames[found_index].last_access_time = time;
    } else {
        // Page fault
        printf("  x Page Fault! Page %d tidak ada di memori\n", page);
        page_faults++;

        // Find empty frame or replace
        int target_frame = -1;
        for (int i = 0; i < num_frames; i++) {
            if (frames[i].page_number == -1) {
                target_frame = i;
                printf("  → Menggunakan frame kosong %d\n", i);
                break;
            }
        }
    }
}

```



```

        if (target_frame == -1) {
            // Need replacement
            printf(" → Memori penuh! Perlu page replacement\n");

            if (algo_type == 1) {
                target_frame = fifo_replacement(frames, num_frames,
time);
            } else if (algo_type == 2) {
                target_frame = lru_replacement(frames, num_frames,
time);
            } else if (algo_type == 3) {
                target_frame = clock_replacement(frames, num_frames,
&clock_hand);
            }
        }

        // Load page
        frames[target_frame].page_number = page;
        frames[target_frame].referenced = 1;
        frames[target_frame].modified = 0;
        frames[target_frame].load_time = time;
        frames[target_frame].last_access_time = time;
    }

    print_frames(frames, num_frames);
}

printf("\n\n===== HASIL %s =====\n", name);
printf("Total Page Faults: %d\n", page_faults);
printf("Hit Rate: %.2f%%\n",
    ((seq_length - page_faults) * 100.0) / seq_length);
printf("=====\n");
}

int main() {
    printf("=== SIMULASI PAGE REPLACEMENT ALGORITHMS ===\n\n");

    // Test case 1
    int test1[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    int len1 = sizeof(test1) / sizeof(test1[0]);
    int frames = 3;

    printf("Test Case:\n");
    printf("Sequence: ");
    for (int i = 0; i < len1; i++) {
        printf("%d ", test1[i]);
    }
    printf("\nFrames: %d\n", frames);

    // FIFO
    simulate_algorithm("FIFO", test1, len1, frames, 1);
    printf("\nTekan Enter untuk lanjut ke LRU...\n");
    getchar();
}

```

```
// LRU
simulate_algorithm("LRU", test1, len1, frames, 2);
printf("\nTekan Enter untuk lanjut ke Clock...\n");
getchar();

// Clock
simulate_algorithm("CLOCK/Second Chance", test1, len1, frames, 3);

// Belady's Anomaly
printf("\n\n=== DEMONSTRASI BELADY'S ANOMALY ===\n");
printf("FIFO dapat mengalami lebih banyak page faults dengan lebih
banyak frames!\n");

printf("\nTekan Enter untuk simulasi dengan 3 frames...\n");
getchar();
simulate_algorithm("FIFO (3 frames)", test1, len1, 3, 1);

printf("\nTekan Enter untuk simulasi dengan 4 frames...\n");
getchar();
simulate_algorithm("FIFO (4 frames)", test1, len1, 4, 1);

printf("\n=== KESIMPULAN ===\n");
printf("1. LRU biasanya menghasilkan page faults paling sedikit\n");
printf("2. FIFO sederhana tapi bisa mengalami Belady's Anomaly\n");
printf("3. Clock adalah kompromi yang baik (efficient + good
performance)\n");
printf("4. Pilihan algoritma tergantung pada access pattern
program\n");

return 0;
}
```

Langkah 3: Compile dan jalankan

```
gcc -o page_replacement page_replacement.c
./page_replacement
```

Langkah 4: Analisis Output

Bandingkan:

- Jumlah page faults untuk setiap algoritma
- Perhatikan Belady's Anomaly pada FIFO
- Algoritma mana yang paling efisien?

Pertanyaan Diskusi:

1. Mengapa LRU lebih baik dari FIFO?
2. Kapan Clock algorithm memberikan hasil mendekati LRU?
3. Bagaimana access pattern mempengaruhi performance algoritma?

5. ALLOCATION OF FRAMES

5.1 Penjelasan Konsep

Sistem operasi harus memutuskan **berapa banyak frame** yang dialokasikan untuk setiap proses. Alokasi yang buruk dapat menyebabkan thrashing.

Strategi Alokasi:

1. Equal Allocation

- Semua proses mendapat jumlah frame yang sama
- Sederhana tapi tidak adil
- Formula: $\text{frames_per_process} = \text{total_frames} / \text{num_processes}$

2. Proportional Allocation

- Frame dialokasikan proporsional dengan ukuran proses
- Lebih adil dari equal allocation
- Formula: $\text{allocated_i} = (\text{size_i} / \text{total_size}) * \text{total_frames}$

3. Priority-Based Allocation

- Frame dialokasikan berdasarkan prioritas proses
- Proses penting mendapat lebih banyak frame
- Formula: $\text{allocated_i} = (\text{priority_i} / \text{total_priority}) * \text{total_frames}$

Minimum Frames: Setiap proses harus mendapat minimum frames untuk menghindari thrashing. Minimum tergantung pada:

- Instruction set architecture (beberapa instruksi bisa akses banyak page)
- Working set size proses

5.2 Praktik: Simulasi Frame Allocation

Langkah 1: Buat program

```
cd ~/praktikum-vm/frames
nano frame_allocation.c
```

Langkah 2: Ketik kode lengkap

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_PROCESSES 10

typedef struct {
```

```

    int pid;
    int size;
    int priority;
    int allocated_frames;
    int page_faults;
} Process;

void print_processes(Process processes[], int num_processes) {
    printf("\n%-5s | %-10s | %-10s | %-15s | %-12s\n",
           "PID", "Size(pages)", "Priority", "Alloc Frames", "Page
Faults");
    printf("-----|-----|-----|-----|-----
--\n");

    for (int i = 0; i < num_processes; i++) {
        printf("%-5d | %-10d | %-10d | %-15d | %-12d\n",
               processes[i].pid,
               processes[i].size,
               processes[i].priority,
               processes[i].allocated_frames,
               processes[i].page_faults);
    }
    printf("\n");
}

void equal_allocation(Process processes[], int num_processes, int
total_frames) {
    printf("\n=== EQUAL ALLOCATION ===\n");
    printf("Total Frames: %d\n", total_frames);
    printf("Number of Processes: %d\n", num_processes);

    int frames_per_process = total_frames / num_processes;
    int remaining = total_frames % num_processes;

    printf("Frames per process: %d\n", frames_per_process);
    printf("Remaining frames: %d\n\n", remaining);

    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated_frames = frames_per_process;
        if (i < remaining) {
            processes[i].allocated_frames++;
        }
    }

    print_processes(processes, num_processes);
}

void proportional_allocation(Process processes[], int num_processes, int
total_frames) {
    printf("\n=== PROPORTIONAL ALLOCATION ===\n");
    printf("Total Frames: %d\n", total_frames);

    int total_size = 0;
    for (int i = 0; i < num_processes; i++) {

```

```

        total_size += processes[i].size;
    }

    printf("Total Size (pages): %d\n\n", total_size);

    int allocated = 0;
    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated_frames =
            (processes[i].size * total_frames) / total_size;
        allocated += processes[i].allocated_frames;
    }

    // Distribute remaining
    int remaining = total_frames - allocated;
    for (int i = 0; i < remaining && i < num_processes; i++) {
        processes[i].allocated_frames++;
    }

    print_processes(processes, num_processes);
}

void priority_allocation(Process processes[], int num_processes, int
total_frames) {
    printf("\n=== PRIORITY-BASED ALLOCATION ===\n");
    printf("Total Frames: %d\n", total_frames);

    int total_priority = 0;
    for (int i = 0; i < num_processes; i++) {
        total_priority += processes[i].priority;
    }

    printf("Total Priority Weight: %d\n\n", total_priority);

    int allocated = 0;
    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated_frames =
            (processes[i].priority * total_frames) / total_priority;

        // Minimum 2 frames per process
        if (processes[i].allocated_frames < 2) {
            processes[i].allocated_frames = 2;
        }
        allocated += processes[i].allocated_frames;
    }

    // Adjust if over-allocated
    while (allocated > total_frames) {
        for (int i = num_processes - 1; i >= 0 && allocated >
total_frames; i--) {
            if (processes[i].allocated_frames > 2) {
                processes[i].allocated_frames--;
                allocated--;
            }
        }
    }
}

```

```

    }

    // Distribute remaining
    while (allocated < total_frames) {
        for (int i = 0; i < num_processes && allocated < total_frames;
i++) {
            processes[i].allocated_frames++;
            allocated++;
        }
    }

    print_processes(processes, num_processes);
}

void simulate_page_faults(Process processes[], int num_processes) {
    printf("\n=== SIMULASI PAGE FAULTS ===\n");
    printf("Mensimulasikan working set untuk setiap proses...\n\n");

    for (int i = 0; i < num_processes; i++) {
        int working_set_size = processes[i].size / 2;

        if (processes[i].allocated_frames >= working_set_size) {
            processes[i].page_faults = 10 + rand() % 20;
        } else {
            int shortage = working_set_size -
processes[i].allocated_frames;
            processes[i].page_faults = 30 + (shortage * 10) + rand() % 30;
        }
    }

    print_processes(processes, num_processes);

    int total_faults = 0;
    for (int i = 0; i < num_processes; i++) {
        total_faults += processes[i].page_faults;
    }
    printf("Total Page Faults: %d\n", total_faults);
}

int main() {
    printf("=== SIMULASI FRAME ALLOCATION STRATEGIES ===\n\n");

    int num_processes = 5;
    int total_frames = 64;

    Process processes[MAX_PROCESSES] = {
        {1, 100, 5, 0, 0},
        {2, 200, 3, 0, 0},
        {3, 50, 8, 0, 0},
        {4, 150, 4, 0, 0},
        {5, 80, 6, 0, 0}
    };

    printf("Proses yang akan dialokasikan:\n");

```

```

print_processes(processes, num_processes);

printf("\n\nTekan Enter untuk Equal Allocation...\n");
getchar();
equal_allocation(processes, num_processes, total_frames);
simulate_page_faults(processes, num_processes);

printf("\n\nTekan Enter untuk Proportional Allocation...\n");
getchar();
proportional_allocation(processes, num_processes, total_frames);
simulate_page_faults(processes, num_processes);

printf("\n\nTekan Enter untuk Priority Allocation...\n");
getchar();
priority_allocation(processes, num_processes, total_frames);
simulate_page_faults(processes, num_processes);

printf("\n\n=== PERBANDINGAN ===\n");
printf("Strategi mana yang menghasilkan page faults paling sedikit?
\n");
printf("Pertimbangkan:\n");
printf("- Equal: Sederhana tapi tidak mempertimbangkan kebutuhan
proses\n");
printf("- Proportional: Mempertimbangkan ukuran proses\n");
printf("- Priority: Mempertimbangkan prioritas proses\n");

return 0;
}

```

Langkah 3: Compile dan jalankan

```

gcc -o frame_allocation frame_allocation.c
./frame_allocation

```

Langkah 4: Analisis

Perhatikan:

- Bagaimana frame didistribusikan pada setiap strategi
- Strategi mana yang menghasilkan page faults terendah
- Tradeoff antara fairness dan efficiency

6. THRASHING

6.1 Penjelasan Konsep

Thrashing terjadi ketika sistem menghabiskan lebih banyak waktu untuk **paging** (menukar halaman) daripada **mengeksekusi program**.

Penyebab Thrashing:

1. Degree of multiprogramming terlalu tinggi (terlalu banyak proses)
2. Setiap proses mendapat terlalu sedikit frame
3. Working set tidak muat di allocated frames
4. Page fault rate sangat tinggi

Tanda-tanda Thrashing:

- CPU utilization rendah
- Disk I/O sangat tinggi
- Sistem sangat lambat/tidak responsif
- Swap usage tinggi

Solusi:

1. **Working Set Model**: Alokasikan frame sesuai working set
2. **Page Fault Frequency (PFF)**: Monitor page fault rate
3. **Suspend proses**: Kurangi degree of multiprogramming
4. **Tambah RAM**: Solusi hardware

6.2 Praktik: Simulasi Thrashing

Langkah 1: Buat program thrashing demo

```
cd ~/praktikum-vm/thrashing
nano thrashing_demo.c
```

Langkah 2: Ketik kode (PERINGATAN: Dapat membuat sistem lambat!)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>
#include <sys/resource.h>

#define MB (1024 * 1024)
#define ALLOC_SIZE (10 * MB)

void print_memory_stats() {
    FILE* fp = fopen("/proc/meminfo", "r");
```



```

    if (!fp) return;

    char line[256];
    long mem_total = 0, mem_free = 0, mem_available = 0;
    long swap_total = 0, swap_free = 0;

    while (fgets(line, sizeof(line), fp)) {
        if (sscanf(line, "MemTotal: %ld", &mem_total) == 1) continue;
        if (sscanf(line, "MemFree: %ld", &mem_free) == 1) continue;
        if (sscanf(line, "MemAvailable: %ld", &mem_available) == 1)
            continue;
        if (sscanf(line, "SwapTotal: %ld", &swap_total) == 1) continue;
        if (sscanf(line, "SwapFree: %ld", &swap_free) == 1) continue;
    }

    fclose(fp);

    printf("  Mem Total: %ld MB | Free: %ld MB | Available: %ld MB\n",
           mem_total / 1024, mem_free / 1024, mem_available / 1024);
    printf("  Swap Total: %ld MB | Free: %ld MB | Used: %ld MB\n",
           swap_total / 1024, swap_free / 1024,
           (swap_total - swap_free) / 1024);
}

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void working_set_access(char* memory, size_t size, int iterations) {
    for (int i = 0; i < iterations; i++) {
        int index = rand() % size;
        memory[index] = (char)(i % 256);
        volatile char temp = memory[index];
        (void)temp;
    }
}

void simulate_process(int proc_id, size_t alloc_size, int
access_iterations) {
    printf("\n[Process %d] Mengalokasikan %zu MB...\n",
           proc_id, alloc_size / MB);

    double start = get_time();

    char* memory = (char*)malloc(alloc_size);
    if (!memory) {
        printf("[Process %d] ERROR: Gagal alokasi memori!\n", proc_id);
        return;
    }

    // Inisialisasi (force allocation)
    for (size_t i = 0; i < alloc_size; i += 4096) {

```

```

        memory[i] = 0;
    }

    double alloc_time = get_time() - start;
    printf("[Process %d] Alokasi selesai dalam %.2f detik\n",
        proc_id, alloc_time);

    printf("[Process %d] Mengakses working set (%d iterations)...\n",
        proc_id, access_iterations);

    start = get_time();
    working_set_access(memory, alloc_size, access_iterations);
    double access_time = get_time() - start;

    printf("[Process %d] Akses selesai dalam %.2f detik\n",
        proc_id, access_time);
    printf("[Process %d] Throughput: %.2f accesses/sec\n",
        proc_id, access_iterations / access_time);

    free(memory);
}

void thrashing_simulation(int num_processes) {
    printf("\n=====");
    printf("SIMULASI dengan %d proses\n", num_processes);
    printf("=====\n\n");

    printf("Memory Status SEBELUM simulasi:\n");
    print_memory_stats();

    printf("\n\nMulai simulasi...\n");
    printf("Tekan Ctrl+C jika sistem tidak responsif (thrashing)\n\n");

    double total_start = get_time();

    for (int i = 0; i < num_processes; i++) {
        simulate_process(i + 1, ALLOC_SIZE, 100000);

        printf("\nMemory Status setelah process %d:\n", i + 1);
        print_memory_stats();

        // Check thrashing
        FILE* fp = fopen("/proc/meminfo", "r");
        if (fp) {
            char line[256];
            long swap_total = 0, swap_free = 0;

            while (fgets(line, sizeof(line), fp)) {
                if (sscanf(line, "SwapTotal: %ld", &swap_total) == 1)
                    continue;
                if (sscanf(line, "SwapFree: %ld", &swap_free) == 1)
                    continue;
            }
            fclose(fp);
        }
    }
}

```

```

        long swap_used = swap_total - swap_free;
        if (swap_used > swap_total / 2 && swap_total > 0) {
            printf("\n⚠ WARNING: THRASHING DETECTED!\n");
            printf("    Swap usage sangat tinggi: %ld MB / %ld MB\n",
                swap_used / 1024, swap_total / 1024);
            printf("    Sistem menghabiskan waktu untuk paging!\n");
        }
    }

    printf("\n---\n");
    sleep(1);
}

double total_time = get_time() - total_start;

printf("\n===== \n");
printf("HASIL SIMULASI\n");
printf("===== \n");
printf("Total waktu: %.2f detik\n", total_time);
printf("Rata-rata waktu per proses: %.2f detik\n",
    total_time / num_processes);

printf("\nMemory Status SETELAH simulasi:\n");
print_memory_stats();
}

int main() {
    printf("=== SIMULASI DAN DETEKSI THRASHING ===\n\n");

    srand(time(NULL));

    printf("PERINGATAN: Simulasi ini dapat membuat sistem lambat!\n");
    printf("Pastikan Anda telah menyimpan pekerjaan lain.\n\n");

    printf("Test 1: Normal Load (3 proses)\n");
    printf("Tekan Enter untuk mulai...\n");
    getchar();

    thrashing_simulation(3);

    printf("\n\nTest 1 selesai. Perhatikan:\n");
    printf("- Waktu akses relatif cepat\n");
    printf("- Swap usage minimal\n");
    printf("- Sistem responsif\n");

    printf("\n\nTest 2: Heavy Load - Inducing Thrashing\n");
    printf("PERINGATAN: Test ini dapat menyebabkan thrashing!\n");
    printf("Tekan Enter untuk mulai (atau Ctrl+C untuk skip)...\n");
    getchar();

    thrashing_simulation(8);

    printf("\n\n=== KESIMPULAN ===\n");

```

```

printf("Thrashing terjadi ketika:\n");
printf("1. Terlalu banyak proses berjalan bersamaan\n");
printf("2. Working set tidak muat di physical memory\n");
printf("3. Sistem terus-menerus melakukan page swap\n");
printf("\nSolusi:\n");
printf("1. Kurangi degree of multiprogramming\n");
printf("2. Tambah physical memory\n");
printf("3. Gunakan working set model\n");
printf("4. Suspend proses dengan prioritas rendah\n");

return 0;
}

```

Langkah 3: Compile dan jalankan (HATI-HATI!)

```

gcc -o thrashing_demo thrashing_demo.c
./thrashing_demo

```

PERINGATAN: Program ini dapat membuat sistem sangat lambat!

6.3 Praktik: Working Set Model

Langkah 1: Buat program working set

```

nano working_set.c

```

Langkah 2: Ketik kode

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_PAGES 50
#define MAX_TIME 100

typedef struct {
    int time;
    int page;
} Reference;

void calculate_working_set(Reference refs[], int num_refs, int delta) {
    printf("\n== WORKING SET MODEL ==\n");
    printf("Delta (window size): %d\n\n", delta);

    printf("Time | Page | Working Set\n");
    printf("-----|-----|-----\n");
}

```

```

    for (int t = 0; t < num_refs; t++) {
        bool in_ws[MAX_PAGES] = {false};
        int ws_size = 0;

        for (int i = t; i >= 0 && i > t - delta; i--) {
            if (!in_ws[refs[i].page]) {
                in_ws[refs[i].page] = true;
                ws_size++;
            }
        }

        printf(" %3d | %4d | { ", refs[t].time, refs[t].page);
        for (int p = 0; p < MAX_PAGES; p++) {
            if (in_ws[p]) {
                printf("%d ", p);
            }
        }
        printf("} (size=%d)\n", ws_size);
    }
}

int main() {
    printf("=== WORKING SET MODEL SIMULATION ===\n\n");

    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 1, 2, 3};
    int num_refs = sizeof(pages) / sizeof(pages[0]);

    Reference refs[MAX_TIME];
    for (int i = 0; i < num_refs; i++) {
        refs[i].time = i + 1;
        refs[i].page = pages[i];
    }

    printf("Page Reference String: ");
    for (int i = 0; i < num_refs; i++) {
        printf("%d ", pages[i]);
    }
    printf("\n");

    printf("\n\nTest dengan Delta = 3\n");
    calculate_working_set(refs, num_refs, 3);

    printf("\n\nTekan Enter untuk Delta = 5...\n");
    getchar();
    calculate_working_set(refs, num_refs, 5);

    printf("\n\n=== INTERPRETASI ===\n");
    printf("- Working Set = pages yang diakses dalam delta terakhir\n");
    printf("- Delta kecil → Working set kecil → Butuh lebih sedikit frames\n");
    printf("- Delta besar → Working set besar → Butuh lebih banyak frames\n");
    printf("- Jika allocated frames < working set size → THRASHING!\n");
}

```

```
    return 0;  
}
```

Langkah 3: Compile dan jalankan

```
gcc -o working_set working_set.c  
./working_set
```

7. MEMORY COMPRESSION

7.1 Penjelasan Konsep

Memory Compression adalah teknik untuk mengurangi kebutuhan swap dengan **mengkompresi** page yang jarang digunakan di dalam RAM, daripada swap ke disk.

Proses:

1. Page jarang digunakan diidentifikasi
2. Page dikompresi in-memory
3. Space yang dihemat bisa digunakan untuk page lain
4. Saat diakses, page didekompresi

Keuntungan:

- Lebih cepat dari disk I/O (10-100x)
- Mengurangi wear pada SSD
- Efektif untuk data yang mudah dikompresi

Kerugian:

- CPU overhead untuk compress/decompress
- Tidak semua data bisa dikompresi dengan baik
- Membutuhkan space untuk decompression

Digunakan oleh:

- macOS (Memory Compression)
- Linux (zswap, zram)
- Windows 10+ (Memory Compression)

7.2 Praktik: Memory Compression Simulation

Langkah 1: Install library zlib

```
sudo apt-get install zlib1g-dev
```

Langkah 2: Buat program

```
cd ~/praktikum-vm/compression
nano memory_compression.c
```

Langkah 3: Ketik kode lengkap

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <zlib.h>

#define PAGE_SIZE 4096
#define NUM_PAGES 100

typedef struct {
    int page_id;
    int is_compressed;
    size_t original_size;
    size_t compressed_size;
    void* data;
} Page;

void generate_compressible_data(char* buffer, size_t size) {
    char pattern[] = "ABCDEFGH";
    for (size_t i = 0; i < size; i++) {
        buffer[i] = pattern[i % 8];
    }
}

void generate_random_data(char* buffer, size_t size) {
    for (size_t i = 0; i < size; i++) {
        buffer[i] = rand() % 256;
    }
}

int compress_page(Page* page) {
    if (page->is_compressed) return 0;

    uLongf compressed_size = compressBound(page->original_size);
    Bytef* compressed_data = (Bytef*)malloc(compressed_size);

    if (!compressed_data) {
        printf("Error: Malloc failed\n");
        return -1;
    }

    int result = compress(compressed_data, &compressed_size,
                          (Bytef*)page->data, page->original_size);
```

```
    if (result != Z_OK) {
        printf("Error: Compression failed\n");
        free(compressed_data);
        return -1;
    }

    free(page->data);
    page->data = compressed_data;
    page->compressed_size = compressed_size;
    page->is_compressed = 1;

    return 0;
}

int decompress_page(Page* page) {
    if (!page->is_compressed) return 0;

    Bytef* decompressed_data = (Bytef*)malloc(page->original_size);
    if (!decompressed_data) {
        printf("Error: Malloc failed\n");
        return -1;
    }

    uLongf decompressed_size = page->original_size;
    int result = uncompress(decompressed_data, &decompressed_size,
                           (Bytef*)page->data, page->compressed_size);

    if (result != Z_OK) {
        printf("Error: Decompression failed\n");
        free(decompressed_data);
        return -1;
    }

    free(page->data);
    page->data = decompressed_data;
    page->is_compressed = 0;

    return 0;
}

void print_page_stats(Page pages[], int num_pages) {
    size_t total_original = 0;
    size_t total_compressed = 0;
    int compressed_count = 0;

    for (int i = 0; i < num_pages; i++) {
        total_original += pages[i].original_size;
        if (pages[i].is_compressed) {
            total_compressed += pages[i].compressed_size;
            compressed_count++;
        } else {
            total_compressed += pages[i].original_size;
        }
    }
}
```



```

    }

    printf("\n===== MEMORY STATISTICS =====\n");
    printf("Total Pages: %d\n", num_pages);
    printf("Compressed Pages: %d\n", compressed_count);
    printf("Original Total Size: %zu KB\n", total_original / 1024);
    printf("Current Total Size: %zu KB\n", total_compressed / 1024);
    printf("Memory Saved: %zu KB (%.1f%%)\n",
           (total_original - total_compressed) / 1024,
           ((total_original - total_compressed) * 100.0) /
total_original);
    printf("Compression Ratio: %.2f:1\n",
           (double)total_original / total_compressed);
    printf("===== \n");
}

int main() {
    printf("=== SIMULASI MEMORY COMPRESSION ===\n\n");

    srand(time(NULL));

    Page pages[NUM_PAGES];

    printf("Membuat %d pages...\n", NUM_PAGES);
    for (int i = 0; i < NUM_PAGES; i++) {
        pages[i].page_id = i;
        pages[i].original_size = PAGE_SIZE;
        pages[i].compressed_size = 0;
        pages[i].is_compressed = 0;
        pages[i].data = malloc(PAGE_SIZE);

        if (i < NUM_PAGES / 2) {
            generate_compressible_data(pages[i].data, PAGE_SIZE);
        } else {
            generate_random_data(pages[i].data, PAGE_SIZE);
        }
    }

    printf("Pages created!\n");
    print_page_stats(pages, NUM_PAGES);

    printf("\n\nTekan Enter untuk kompresi pages...\n");
    getchar();

    printf("\nMengkompresi pages...\n");
    clock_t start = clock();

    for (int i = 0; i < NUM_PAGES; i++) {
        compress_page(&pages[i]);
        if ((i + 1) % 10 == 0) {
            printf("  Progress: %d/%d pages compressed\n", i + 1,
NUM_PAGES);
        }
    }
}

```

```

clock_t end = clock();
double compression_time = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("\nKompresi selesai dalam %.3f detik\n", compression_time);
print_page_stats(pages, NUM_PAGES);

printf("\n\nTekan Enter untuk dekompresi beberapa pages...\n");
getchar();

printf("\nMendekompresi 10 pages pertama...\n");
start = clock();

for (int i = 0; i < 10; i++) {
    decompress_page(&pages[i]);
}

end = clock();
double decompression_time = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("Dekompresi selesai dalam %.3f detik\n", decompression_time);
print_page_stats(pages, NUM_PAGES);

printf("\n\n=== ANALISIS ===\n");
printf("1. Compressible data (pattern) menghasilkan ratio tinggi\n");
printf("2. Random data sulit dikompresi\n");
printf("3. Compression lebih lambat dari decompression\n");
printf("4. Tradeoff: CPU time vs Memory saved vs Disk I/O\n");

printf("\nPerbandingan dengan Swapping:\n");
size_t swap_size = NUM_PAGES * PAGE_SIZE;
size_t compressed_total = 0;
for (int i = 0; i < NUM_PAGES; i++) {
    compressed_total += pages[i].is_compressed ?
                        pages[i].compressed_size :
pages[i].original_size;
}

printf("- Swap ke disk: %zu KB, waktu ~100ms (disk I/O)\n",
    swap_size / 1024);
printf("- Compression: %zu KB, waktu %.3f sec (CPU)\n",
    compressed_total / 1024, compression_time);
printf("- Compression %s untuk kasus ini\n",
    compression_time < 0.1 ? "LEBIH CEPAT" : "lebih lambat");

for (int i = 0; i < NUM_PAGES; i++) {
    free(pages[i].data);
}

return 0;
}

```

```
gcc -o memory_compression memory_compression.c -lz  
./memory_compression
```

Analisis:

- Berapa banyak memori yang dihemat?
- Jenis data apa yang paling efektif dikompresi?
- Kapan sebaiknya menggunakan compression vs swap?

8. ALLOCATING KERNEL MEMORY

8.1 Penjelasan Konsep

Kernel membutuhkan alokasi memori yang berbeda dari user processes:

- Kernel membutuhkan memori **physically contiguous**
- Harus mendukung berbagai ukuran alokasi (small & large)
- Perlu sangat efisien (no page faults allowed!)

Algoritma Kernel Memory Allocation:

1. Buddy System

- Alokasi dalam ukuran power-of-2
- Mudah melakukan coalescing
- Internal fragmentation bisa tinggi

2. Slab Allocator

- Cache untuk objek kernel yang sering digunakan
- Pre-initialize objek
- Sangat efisien untuk fixed-size objects

8.2 Praktik: Buddy System Simulation

Langkah 1: Buat program

```
cd ~/praktikum-vm/kernel-alloc  
nano buddy_system.c
```

Langkah 2: Ketik kode buddy system

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <math.h>
```

```

#define MAX_ORDER 10
#define MIN_SIZE 4 // KB

typedef struct Block {
    int size;
    int is_free;
    int order;
    struct Block* next;
} Block;

typedef struct {
    Block* free_lists[MAX_ORDER];
    int total_size;
} BuddySystem;

void init_buddy_system(BuddySystem* bs, int total_size_kb) {
    bs->total_size = total_size_kb;

    for (int i = 0; i < MAX_ORDER; i++) {
        bs->free_lists[i] = NULL;
    }

    int max_order = (int)(log2(total_size_kb / MIN_SIZE));
    if (max_order >= MAX_ORDER) max_order = MAX_ORDER - 1;

    Block* initial = (Block*)malloc(sizeof(Block));
    initial->size = MIN_SIZE * (1 << max_order);
    initial->is_free = 1;
    initial->order = max_order;
    initial->next = NULL;

    bs->free_lists[max_order] = initial;

    printf("Buddy System initialized with %d KB\n", initial->size);
    printf("Max Order: %d, Block Size: %d KB\n\n", max_order, initial->size);
}

int calculate_order(int size_kb) {
    int order = 0;
    int block_size = MIN_SIZE;

    while (block_size < size_kb && order < MAX_ORDER - 1) {
        block_size *= 2;
        order++;
    }

    return order;
}

void print_free_lists(BuddySystem* bs) {
    printf("\n===== FREE LISTS =====\n");
    printf("Order | Size (KB) | Blocks\n");
}

```

```

printf("-----|-----|-----\n");

for (int i = 0; i < MAX_ORDER; i++) {
    int size = MIN_SIZE * (1 << i);
    int count = 0;
    Block* current = bs->free_lists[i];

    while (current) {
        count++;
        current = current->next;
    }

    if (count > 0) {
        printf("  %2d |   %6d | %d blocks\n", i, size, count);
    }
}
printf("=====\n\n");
}

Block* split_block(BuddySystem* bs, int order) {
    if (order < 0 || order >= MAX_ORDER) {
        return NULL;
    }

    if (bs->free_lists[order] == NULL) {
        if (order + 1 >= MAX_ORDER) {
            printf("  ERROR: No blocks available!\n");
            return NULL;
        }

        Block* larger = split_block(bs, order + 1);
        if (larger == NULL) {
            return NULL;
        }

        printf("  → Splitting %d KB block into two %d KB blocks\n",
            larger->size, larger->size / 2);

        Block* buddy = (Block*)malloc(sizeof(Block));
        buddy->size = larger->size / 2;
        buddy->is_free = 1;
        buddy->order = order;
        buddy->next = bs->free_lists[order];

        larger->size = larger->size / 2;
        larger->order = order;

        bs->free_lists[order] = buddy;

        return larger;
    }

    Block* block = bs->free_lists[order];
    bs->free_lists[order] = block->next;
}

```

```

    block->next = NULL;

    return block;
}

Block* allocate_buddy(BuddySystem* bs, int size_kb) {
    printf("\n>>> ALLOCATE %d KB\n", size_kb);

    int order = calculate_order(size_kb);
    int actual_size = MIN_SIZE * (1 << order);

    printf("  Requested: %d KB → Order: %d, Actual Size: %d KB\n",
           size_kb, order, actual_size);

    Block* block = split_block(bs, order);

    if (block == NULL) {
        printf("  ✗ ALLOCATION FAILED!\n");
        return NULL;
    }

    block->is_free = 0;
    printf("  ✓ Allocated %d KB block (Order %d)\n", block->size, block->order);

    if (actual_size > size_kb) {
        printf("  △ Internal fragmentation: %d KB wasted\n",
               actual_size - size_kb);
    }

    return block;
}

void free_buddy(BuddySystem* bs, Block* block) {
    if (block == NULL) return;

    printf("\n>>> FREE %d KB block (Order %d)\n", block->size, block->order);

    block->is_free = 1;
    block->next = bs->free_lists[block->order];
    bs->free_lists[block->order] = block;

    printf("  ✓ Block freed\n");
}

int main() {
    printf("=== BUDDY SYSTEM ALLOCATION SIMULATOR ===\n\n");

    BuddySystem bs;
    init_buddy_system(&bs, 1024); // 1 MB

    print_free_lists(&bs);
}

```

```

printf("Tekan Enter untuk mulai alokasi...\n");
getchar();

Block* b1 = allocate_buddy(&bs, 100);
print_free_lists(&bs);
printf("Tekan Enter...\n");
getchar();

Block* b2 = allocate_buddy(&bs, 50);
print_free_lists(&bs);
printf("Tekan Enter...\n");
getchar();

Block* b3 = allocate_buddy(&bs, 200);
print_free_lists(&bs);
printf("Tekan Enter...\n");
getchar();

free_buddy(&bs, b1);
print_free_lists(&bs);
printf("Tekan Enter...\n");
getchar();

free_buddy(&bs, b2);
print_free_lists(&bs);
printf("Tekan Enter...\n");
getchar();

free_buddy(&bs, b3);
print_free_lists(&bs);

printf("\n\n=== KARAKTERISTIK BUDDY SYSTEM ===\n");
printf("Kelebihan:\n");
printf("- Coalescing cepat (buddy berdekatan)\n");
printf("- Fragmentation eksternal rendah\n");
printf("- Algoritma sederhana\n\n");
printf("Kekurangan:\n");
printf("- Internal fragmentation (round up to power of 2)\n");
printf("- Tidak efisien untuk ukuran arbitrary\n");

return 0;
}

```

Langkah 3: Compile dan jalankan

```

gcc -o buddy_system buddy_system.c -lm
./buddy_system

```

8.3 Praktik: Melihat Slab Allocator di Linux

Langkah 1: Periksa slab info

```
# Melihat slab caches
sudo cat /proc/slabinfo | head -20

# Real-time monitoring
sudo apt-get install sysstat
sudo slabtop
```

Langkah 2: Buat program untuk parse slab info

```
nano slab_info.c
```

Langkah 3: Ketik kode

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[64];
    int active_objs;
    int num_objs;
    int objsize;
} SlabInfo;

void parse_slabinfo() {
    FILE* fp = fopen("/proc/slabinfo", "r");
    if (!fp) {
        perror("Error opening /proc/slabinfo");
        printf("Try: sudo ./slab_info\n");
        return;
    }

    char line[256];
    fgets(line, sizeof(line), fp);
    fgets(line, sizeof(line), fp);

    printf("\n===== SLAB ALLOCATOR INFO =====\n");
    printf("%-20s | Active | Total | ObjSize\n", "Cache Name");
    printf("-----|-----|-----|-----\n");

    SlabInfo slabs[50];
    int count = 0;

    while (fgets(line, sizeof(line), fp) && count < 50) {
```



```

    SlabInfo info;
    int pagesperslab;

    if (sscanf(line, "%63s %d %d %d %d %d",
               info.name,
               &info.active_objs,
               &info.num_objs,
               &info.objsize,
               &pagesperslab,
               &pagesperslab) >= 4) {

        slabs[count++] = info;

        printf("%-20s | %6d | %5d | %7d\n",
               info.name,
               info.active_objs,
               info.num_objs,
               info.objsize);
    }
}

fclose(fp);

long total_memory = 0;
long active_memory = 0;

for (int i = 0; i < count; i++) {
    total_memory += (long)slabs[i].num_objs * slabs[i].objsize;
    active_memory += (long)slabs[i].active_objs * slabs[i].objsize;
}

printf("\n===== STATISTICS =====\n");
printf("Total Slab Caches: %d\n", count);
printf("Total Memory: %.2f MB\n", total_memory / (1024.0 * 1024.0));
printf("Active Memory: %.2f MB\n", active_memory / (1024.0 * 1024.0));
printf("Utilization: %.1f%%\n",
       (active_memory * 100.0) / total_memory);
printf("=====\n");

printf("\n=== PENJELASAN ===\n");
printf("Slab Allocator digunakan kernel untuk:\n");
printf("- Mengalokasikan objek kernel yang sering digunakan\n");
printf("- Mengurangi overhead alokasi/dealokasi\n");
printf("- Menjaga objek siap pakai (pre-initialized)\n");
printf("\nContoh slab caches:\n");
printf("- dentry: Directory entries\n");
printf("- inode_cache: Inode structures\n");
printf("- task_struct: Process descriptors\n");
}

int main() {
    printf("=== LINUX SLAB ALLOCATOR INFORMATION ===\n");
    parse_slabinfo();
}

```

```
    return 0;  
}
```

Langkah 4: Compile dan jalankan

```
gcc -o slab_info slab_info.c  
sudo ./slab_info
```

Mulai pertemuan ini tidak ada tugas, hanya pengumpulan laporan hasil praktikum saja karena mulai pekan ini, tugas akan difokuskan terhadap pembuatan proyek akhir yaitu membuat sistem operasi linux sendiri. Detailnya akan saya sampaikan ketika perkuliahan daring.